Lab 7: Fuzzing for Input Validation Bugs

Smart Thermostat Controller — `thermostat_control.py` (repo L6_L7_29_thermostat_control, lab7) · Tool: Hypothesis 6.170 + pytest

In this repo the application is `thermostat_control.py` and the test file is `test_thermostat_control.py` (the handout's `processor.py` / `test_processor.py`). The test file fuzzes all three functions with `st.text() | st.none()` ("must never crash") and adds property tests with generated valid and invalid inputs (integers, floats, lists, times) to check the answers are actually correct.

Step 2 — Running the buggy program

With a plain number `70` the program silently drops the last digit (7.0 °C), and minutes are formatted inconsistently (`day 5`):

Step 2 — buggy program, normal-looking input (wrong results)

```
$ python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): 70
Setpoint in Celsius: 7.0

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5): 20.5,21,22.5
Average Temperature: 21.333333333333332

==== Running schedule_slot ====
Enter a time as HH:MM (e.g. 07:30): 07:05
Schedule Slot: day 5
```

Bad input (`abc`, an empty CSV value) crashes the program with an unhandled `ValueError`:

Step 2 — buggy program crashing on bad input

```
$ python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): abc
Traceback (most recent call last):
  File ".../lab7/thermostat_control.py", line 35, in <module>
    main()
    ~~~~^^
  File ".../lab7/thermostat_control.py", line 21, in main
    result = parse_setpoint(user_input)
  File ".../lab7/thermostat_control.py", line 2, in parse_setpoint
    value = float(text[:-1])
ValueError: could not convert string to float: 'ab'

$ python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): 21.5C
Setpoint in Celsius: 21.5

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5): 20.5,,22
Traceback (most recent call last):
  File ".../lab7/thermostat_control.py", line 35, in <module>
    main()
    ~~~~^^
  File ".../lab7/thermostat_control.py", line 26, in main
    result = average_temperature(user_input)
  File ".../lab7/thermostat_control.py", line 8, in average_temperature
    values = [float(r) for r in readings_csv.split(',')]
              ~~~~~^^^
ValueError: could not convert string to float: ''
```

Step 4 — Hypothesis tests on the buggy code (failing)

Step 4 — Hypothesis tests on the buggy code: 9 failed

```
$ python -m pytest test_thermostat_control.py --tb=no
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0
rootdir: .../lab7
plugins: platformdirs-4.13.5, anyio-4.15.1, hypothesis-6.170.15
collected 13 items

test_thermostat_control.py FF..FF.F.FFFF                                 [100%]

=========================== short test summary info ============================
FAILED test_thermostat_control.py::test_parse_setpoint_no_crash - ExceptionGr...
FAILED test_thermostat_control.py::test_parse_setpoint_plain_number_is_celsius
FAILED test_thermostat_control.py::test_parse_setpoint_rejects_nan_and_inf - ...
FAILED test_thermostat_control.py::test_average_temperature_no_crash - Except...
FAILED test_thermostat_control.py::test_average_rejects_impossible_readings
FAILED test_thermostat_control.py::test_average_rejects_bad_csv - ExceptionGr...
FAILED test_thermostat_control.py::test_schedule_slot_no_crash - ExceptionGro...
FAILED test_thermostat_control.py::test_schedule_slot_valid_times - Assertion...
FAILED test_thermostat_control.py::test_schedule_slot_rejects_out_of_range - ...
========================= 9 failed, 4 passed in 0.36s ==========================
```

Falsifying examples Hypothesis found and shrank to the smallest failing input (end of the full output):

Step 4 — falsifying examples found by Hypothesis (end of output)

```
$ python -m pytest test_thermostat_control.py --tb=line
... (earlier failures scrolled up) ...
    | assert False
    | Failing test case: test_schedule_slot_no_crash(
    |     s=None,
    | )
    +------------------------------------
.../lab7/test_thermostat_control.py:93: ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
E   AssertionError: assert 'day 0' == 'day 00'

      - day 00
      ?      -
      + day 0
    Failing test case: test_schedule_slot_valid_times(
        hour=6,
        minute=0,
    )
.../lab7/test_thermostat_control.py:104: AssertionError: assert 'day 0' == 'day 00'
E   AssertionError: assert 'day 0' is None
     +  where 'day 0' = schedule_slot('24:00')
    Failing test case: test_schedule_slot_rejects_out_of_range(
        hm=(24, 0),
    )
.../lab7/test_thermostat_control.py:115: AssertionError: assert 'day 0' is None
========================== short test summary info ===========================
FAILED test_thermostat_control.py::test_parse_setpoint_no_crash - ExceptionGr...
FAILED test_thermostat_control.py::test_parse_setpoint_plain_number_is_celsius
FAILED test_thermostat_control.py::test_parse_setpoint_rejects_nan_and_inf - ...
FAILED test_thermostat_control.py::test_average_temperature_no_crash - Except...
FAILED test_thermostat_control.py::test_average_rejects_impossible_readings
FAILED test_thermostat_control.py::test_average_rejects_bad_csv - ExceptionGr...
FAILED test_thermostat_control.py::test_schedule_slot_no_crash - ExceptionGro...
FAILED test_thermostat_control.py::test_schedule_slot_valid_times - Assertion...
FAILED test_thermostat_control.py::test_schedule_slot_rejects_out_of_range - ...
======================== 9 failed, 4 passed in 0.38s =========================
```

Bugs found by fuzzing and how they were fixed

Function	Failing input	Problem	Fix
`parse_setpoint`	`None`	TypeError: 'NoneType' object is not subscriptable	Return None for non-string input
`parse_setpoint`	`'' , 'abc'`	ValueError (float of '' / 'ab')	Strip input; catch ValueError; return None
`parse_setpoint`	`'70' (no unit)`	Silently wrong: last digit cut off, gives 7.0	If last char is not C/F, treat whole text as Celsius → 70.0
`parse_setpoint`	`'nanC', 'infF'`	Returns nan / inf as a temperature	Reject non-finite values (math.isfinite)
`average_temperature`	`None`	AttributeError: no attribute 'split'	Return None for non-string input
`average_temperature`	`'', '20,,21'`	ValueError: could not convert string to float: ''	Validate every value; any bad value → None
`average_temperature`	`'nan', 'INFINITY'`	Returns nan / inf	Reject non-finite readings
`average_temperature`	`'1001.0', 1.8e308 x3`	Impossible reading accepted; huge values overflow the sum	Sanity limit: readings outside ±1000 rejected; use math.fsum

`schedule_slot`	`None`	AttributeError: no attribute 'split'	Return None for non-string input
`schedule_slot`	`'', ':', '7'`	ValueError (unpack / int of '')	Validate with regex [0-9]{1,2}:[0-9]{2}
`schedule_slot`	`'24:00', '25:61'`	Accepted as 'day 0' / 'day 61'	Check 0 ≤ hour ≤ 23 and 0 ≤ minute ≤ 59
`schedule_slot`	`'05:07' vs '07:05'`	Inconsistent: 'night 07' but 'day 5'	Both branches use 2-digit minutes: 'night 07', 'day 05'

All three functions now return `None` for anything invalid, and `main()` prints a clear "Invalid …" message instead of a traceback. Fuzzing also caught a bug in the first version of the fix: three readings of 1.8e308 overflowed the average. That led to the ±1000° sanity limit on sensor readings.

Step 5 — Running the fixed program

Same inputs as before: correct values (70.0, `day 05`), and friendly messages instead of crashes:

Step 5 — fixed program: correct results, no crashes

```
$ python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): 70
Setpoint in Celsius: 70.0

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5): 20.5,21,22.5
Average Temperature: 21.333333333333332

==== Running schedule_slot ====
Enter a time as HH:MM (e.g. 07:30): 07:05
Schedule Slot: day 05

$ python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): 21.5C
Setpoint in Celsius: 21.5

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5): 20.5,,22
Invalid readings. Give numbers separated by commas, e.g. 20.5,21,22.5.

==== Running schedule_slot ====
Enter a time as HH:MM (e.g. 07:30): 25:61
Invalid time. Use HH:MM with hour 0-23 and minute 0-59, e.g. 07:30.

$ python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): abc
Invalid setpoint. Use a number with optional C or F, e.g. 21.5C or 70F.

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5):
Invalid readings. Give numbers separated by commas, e.g. 20.5,21,22.5.

==== Running schedule_slot ====
Enter a time as HH:MM (e.g. 07:30): 07:30
Schedule Slot: day 30
```

Step 6 — Hypothesis tests on the fixed code (passing)

Step 6 — Hypothesis tests on the fixed code: 13 passed

```
$ python -m pytest test_thermostat_control.py
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0
rootdir: .../lab7
plugins: platformdirs-4.13.5, anyio-4.15.1, hypothesis-6.170.15
collected 13 items

test_thermostat_control.py .............                                 [100%]

============================== 13 passed in 1.27s ==============================
```

All 13 tests pass. They were also rerun with 8 different random seeds and with 5,000 examples per test, all passing.